

بسمه تعالی

درس سیستم های نهفته - نیمسال تحصیلی ۰۴-۰۵

دانشگاه صنعتی شریف - دانشکده مهندسی برق



گزارش بخش اول

امیرحسین مطیعی - شماره دانشجویی ۴۰۱۱۰۲۵۵۳

چکیده

در این بخش از تمرین، یک سامانه پایه پایگاه داده توزیع شده برای نگهداری و بازیابی آخرین مقدار ثبت شده سنسورها طراحی و پیاده سازی شد. معماری سامانه از یک نود Master و دو نود Slave تشکیل شده است و هر نود روی یک ماشین مجازی مستقل با سیستم عامل Ubuntu ۲۲.۰۴ اجرا می شود. هر ماشین مجازی پایگاه داده محلی SQLite خود را در اختیار دارد و ارتباط میان نودها از طریق شبکه، نشانی IP و شماره Port انجام می شود. برنامه های نودها با زبان C++ نوشته شده اند و برای پیاده سازی سرویس HTTP از کتابخانه Mongoose استفاده شده است.

در طراحی انجام شده، اپراتور درخواست خود را به نود Master ارسال می کند. Master ابتدا پایگاه داده محلی خود را بررسی می کند. در صورتی که داده سنسور در Master وجود نداشته باشد، درخواست به ترتیب برای Slave_۱ و سپس Slave_۲ ارسال می شود. هر نود Slave تنها پایگاه داده محلی خود را جست و جو می کند و نتیجه را به Master بازمی گرداند. در نهایت Master پاسخ نهایی را در قالب JSON به اپراتور ارائه می دهد. برای ارزیابی سامانه، تست های شبکه، پایگاه داده، کامپایل، اجرای سرویس ها، جست و جوی داده در هر سه نود، مدیریت ورودی نامعتبر، نبود داده، قطع نود و اجرای تست خودکار انجام شد. نتایج نشان دادند که مسیر اصلی درخواست و پاسخ به درستی اجرا می شود، داده ها از نود صحیح بازیابی می شوند و سامانه در شرایط قطع موقت یکی از Slave ها نیز رفتار قابل پیش بینی دارد.

مقدمه

هدف این بخش، آشنایی عملی با طراحی یک سیستم توزیع شده ساده بود که در آن داده ها روی چند نود مستقل نگهداری می شوند و یک نود مرکزی مسئول دریافت درخواست و هماهنگی میان نودها است. در یک سامانه متمرکز، تمام اطلاعات در یک پایگاه داده قرار می گیرند و همه درخواست ها مستقیماً به همان پایگاه داده ارسال می شوند. در مقابل، در معماری توزیع شده داده ها میان چند نود تقسیم می شوند و برنامه باید بتواند محل داده را پیدا کند، خطاهای شبکه را تشخیص دهد و پاسخ نهایی یکپارچه ای به کاربر ارائه دهد.

در این پروژه از معماری Master/Slave استفاده شد. نود Master به عنوان نقطه ورود سامانه در نظر گرفته شد و نودهای Slave به عنوان نگهدارنده بخشی از داده های سنسورها عمل کردند. این ساختار باعث شد اپراتور برای دریافت اطلاعات نیازی به دانستن محل واقعی داده نداشته باشد. از دید اپراتور، فقط یک API وجود دارد، اما در داخل سامانه ممکن است درخواست برای بازیابی داده میان چند ماشین مجازی جابه جا شود.

تحلیل نیازمندی های پروژه

بر اساس صورت تمرین، سامانه باید شامل سه نود مستقل، یعنی یک Master و دو Slave باشد. هر نود باید روی Ubuntu ۲۲.۰۴ اجرا شود، در یک شبکه مشترک قرار گیرد و دارای IP معتبر باشد. ارتباط میان نودها باید بر پایه IP و Port انجام شود و مقادیر ارتباطی نباید به صورت ثابت در متن برنامه نوشته شوند. همچنین هر نود باید یک پایگاه داده محلی SQLite، یک اسکریپت مقداردهی اولیه، یک Makefile و فایل های تنظیمات مستقل داشته باشد.

ورودی برنامه شامل شناسه سنسور و نوع سنسور است و خروجی باید آخرین مقدار ثبت شده برای همان سنسور باشد. رفتار مورد انتظار به این صورت تعریف شد که Master ابتدا پایگاه داده محلی خود را بررسی کند، سپس در صورت نبود داده، Slave۱ را جستجو کند و در ادامه، در صورت نیاز، درخواست را به Slave۲ بفرستد. اگر داده در هیچ نودی وجود نداشت، پاسخ مناسب برای اپراتور تولید شود. علاوه بر رفتار عادی، برنامه باید خطاهای ورودی، خطاهای پایگاه داده، نبود مسیر API، قطع ارتباط با Slave و Timeout را نیز مدیریت کند.

معماری کلی سامانه

معماری پیاده سازی شده شامل چهار جزء منطقی است. جزء اول اپراتور یا سیستم اصلی است که درخواست HTTP را ارسال می کند. جزء دوم نود Master است که درخواست را دریافت کرده و مسئول مدیریت مسیر جستجو است. جزء سوم نود Slave۱ و جزء چهارم نود Slave۲ هستند که هر کدام پایگاه داده محلی مستقلی دارند.

در محیط آزمایش، نشانی نود Master برابر با ۱۹۲.۱۶۸.۱۲۲.۲۲ و Port آن برابر با ۸۰۰۰ بود. نشانی Slave۱ برابر با ۱۹۲.۱۶۸.۱۲۲.۱۸ و Port آن برابر با ۹۰۰۱ و Slave۲ نیز برابر با ۱۹۲.۱۶۸.۱۲۲.۱۹۰ و Port آن برابر با ۹۰۰۲ بود. این مقادیر از فایل های پیکربندی خوانده شدند و در کد برنامه ثابت نبودند.



در این معماری، جستجو به صورت ترتیبی انجام می شود. ترتیب ثابت Master، سپس Slave۱ و سپس Slave۲ باعث می شود رفتار سیستم قابل پیش بینی باشد. مزیت این تصمیم آن است که در صورت وجود داده در Master، هیچ ترافیک شبکه اضافی ایجاد نمی شود. همچنین اگر داده در Slave۱ موجود باشد، نیازی به برقراری ارتباط با Slave۲ وجود ندارد. این روش برای پروژه های با تعداد کم نود مناسب است، هرچند در سامانه های بزرگ تر می توان از روش هایی مانند فهرست محل داده، Hashing یا جستجوی موازی استفاده کرد.

محیط اجرا و روش اتصال به ماشین مجازی

برای اجرای پروژه از سه ماشین مجازی مستقل استفاده شد. هر سه ماشین مجازی در یک شبکه مشترک قرار گرفتند و امکان برقراری ارتباط مستقیم IP میان آن ها فراهم شد. به منظور ساده تر شدن اجرای دستورات و ثبت خروجی ها، به جای باز کردن کنسول گرافیکی هر ماشین مجازی، از سیستم اصلی و پروتکل SSH استفاده شد.

در سیستم اصلی سه پنجره ترمینال جداگانه باز شد. ترمینال اول از طریق SSH به Master، ترمینال دوم به Slave ۱ و ترمینال سوم به Slave ۲ متصل شد. یک ترمینال چهارم نیز روی سیستم اصلی برای ارسال درخواست های curl، بررسی Port ها و اجرای اسکریپت تست در نظر گرفته شد. این روش باعث شد برنامه هر سه نود به صورت هم زمان قابل مشاهده باشد و دستورات بدون نیاز به تایپ مجدد در محیط ماشین های مجازی کپی شوند. علاوه بر راحتی اجرا، استفاده از SSH برای تهیه اسکرین شات های مرتب از خروجی هر نود نیز مناسب بود.

اتصال به ماشین های مجازی با استفاده از دستور ssh :

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ ssh amir@192.168.122.22
amir@192.168.122.22's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-185-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 18 12:43:10 PM UTC 2026

System load:  0.0           Processes:      126
Usage of /:   52.8% of 9.75GB Users logged in: 1
Memory usage: 16%          IPv4 address for enp1s0: 192.168.122.22
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
3 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Jul 18 10:28:49 2026 from 192.168.122.1
```

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ ssh amir@192.168.122.18
amir@192.168.122.18's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-185-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 18 12:43:19 PM UTC 2026

System load:  0.0           Processes:      125
Usage of /:   52.8% of 9.75GB Users logged in: 1
Memory usage: 16%          IPv4 address for enp1s0: 192.168.122.18
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
3 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

Last login: Sat Jul 18 09:52:18 2026 from 192.168.122.22
```

```

amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ ssh amir@192.168.122.190
amir@192.168.122.190's password:
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 5.15.0-185-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Sat Jul 18 12:43:40 PM UTC 2026

System load:  0.0          Processes:            126
Usage of /:   52.8% of 9.75GB Users logged in:      1
Memory usage: 15%         IPv4 address for enp1s0: 192.168.122.190
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

76 updates can be applied immediately.
3 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings.

Last login: Sat Jul 18 09:52:30 2026 from 192.168.122.22

```

مشخصات ماشین های مجازی

```

amir@embedded-master:~$ echo MASTER
MASTER
amir@embedded-master:~$ hostname
embedded-master
amir@embedded-master:~$ hostname -I
192.168.122.22
amir@embedded-master:~$ lsb_release -ds
Ubuntu 22.04.5 LTS
amir@embedded-master:~$ cd ~/amir/master
-bash: cd: /home/amir/amir/master: No such file or directory
amir@embedded-master:~$ ls
master
amir@embedded-master:~$ cd master/
amir@embedded-master:~/master$ cat config
PORT=8000
DATABASE=master.db
SLAVE_TIMEOUT_MS=3000

SLAVE1_IP=192.168.122.18
SLAVE1_PORT=9001

SLAVE2_IP=192.168.122.190
SLAVE2_PORT=9002amir@embedded-master:~/master$

```

```

amir@embedded-slave1:~$ echo SLAVE1
SLAVE1
amir@embedded-slave1:~$ hostname
embedded-slave1
amir@embedded-slave1:~$ hostname -I
192.168.122.18
amir@embedded-slave1:~$ lsb_release -ds
Ubuntu 22.04.5 LTS
amir@embedded-slave1:~$ cd slave1/
amir@embedded-slave1:~/slave1$ cat config
PORT=9001
DATABASE=slave1.db

```

```

amir@embedded-slave2:~$ echo SLAVE2
SLAVE2
amir@embedded-slave2:~$ hostname
embedded-slave2
amir@embedded-slave2:~$ hostname -I
192.168.122.190
amir@embedded-slave2:~$ lsb_release -ds
Ubuntu 22.04.5 LTS
amir@embedded-slave2:~$ cd slave2/
amir@embedded-slave2:~/slave2$ cat config
PORT=9002
DATABASE=slave2.db

```

ابزار ها و وابستگی های نرم افزاری

برنامه های اصلی با زبان C++ نوشته شدند و برای کامپایل آن ها از g++ با استاندارد C++17 استفاده شد. فایل اصلی کتابخانه Mongoose به زبان C است و با gcc کامپایل شد. ابزار make برای اجرای Makefile های هر نود مورد استفاده قرار گرفت. SQLite3 برای ایجاد، مقداردهی، بررسی و خواندن پایگاه داده های محلی استفاده شد. ابزار curl برای ارسال درخواست HTTP و ابزار netcat برای بررسی باز بودن Port ها به کار رفت.

پیش از شروع تست های اصلی، وجود curl، gcc، make، sqlite3 و g++ روی سیستم بررسی شد. خروجی نسخه ابزار ها نشان داد که وابستگی های لازم نصب هستند و محیط برای کامپایل و اجرای پروژه آماده است.

```

amir@embedded-master:~/master$ g++ --version | head -1
g++ (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
amir@embedded-master:~/master$ gcc --version | head -1
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.3) 11.4.0
amir@embedded-master:~/master$ make --version | head -1
GNU Make 4.3
amir@embedded-master:~/master$ sqlite3 --version
3.37.2 2022-01-06 13:25:41 872ba256cbf61d9290b571c0e6d82a20c224ca3ad82971edc46b29818d5dalt1
amir@embedded-master:~/master$ curl --version | head -1
curl 7.81.0 (x86_64-pc-linux-gnu) libcurl/7.81.0 OpenSSL/3.0.2 zlib/1.2.11 brotli/1.0.9 zstd/1.4.8 libidn2/2.3.2 libpsl/0.
21.0 (+libidn2/2.3.2) libssh/0.9.6/openssl/zlib nghttp2/1.43.0 librtmp/2.3 OpenLDAP/2.5.18

```

طراحی اجزای نرم افزاری

برنامه Master

برنامه Master نقش هماهنگ کننده کل سامانه را دارد. این برنامه در زمان اجرا فایل پیکربندی خود را می خواند و اطلاعاتی مانند Port سرویس، مسیر پایگاه داده، IP و Port هر Slave و زمان Timeout را دریافت می کند. سپس یک HTTP Server با استفاده از Mongoose ایجاد می کند و روی ۸۰۰۰ Port منتظر دریافت درخواست می ماند. پس از دریافت درخواست، Master مسیر API و پارامترهای ورودی را اعتبارسنجی می کند. در صورت معتبر بودن ورودی، ابتدا Query مربوط به آخرین مقدار سنسور را روی master.db اجرا می کند. اگر داده پیدا شود، پاسخ مستقیماً با کد ۲۰۰ ارسال می شود. اگر داده محلی وجود نداشته باشد، Master استفاده از HTTP Client پیاده سازی شده در پروژه، درخواست را به Slave۱ ارسال می کند. در صورت دریافت پاسخ ۴۰۴ از Slave۱، همان درخواست به Slave۲ فرستاده می شود. Master پاسخ Slave را دریافت کرده، منبع داده را به پاسخ اضافه می کند و نتیجه نهایی را به اپراتور بازمی گرداند.

برنامه‌های Slave

برنامه Slave در هر دو ماشین Slave ۱ و Slave ۲ ساختار مشابهی دارد. تفاوت اصلی آن‌ها در فایل پیکربندی، شماره Port و مسیر پایگاه داده است. هر Slave یک HTTP Server ایجاد می‌کند و پس از دریافت شناسه و نوع سنسور، فقط پایگاه داده محلی خود را جست‌وجو می‌کند. Slave مسئول تصمیم‌گیری درباره نود بعدی نیست و نتیجه خود را به درخواست‌کننده بازمی‌گرداند. در مسیر اصلی سامانه، درخواست‌کننده Slave باید Master باشد.

کتابخانه Mongoose

Mongoose به‌عنوان لایه ارتباط HTTP استفاده شد. این کتابخانه سبک و رویدادمحور است و امکان پیاده‌سازی HTTP Server و HTTP Client را در برنامه‌های C و C++ فراهم می‌کند. در هر نود، حلقه رویداد Mongoose به‌صورت پیوسته اجرا می‌شود و رویدادهایی مانند دریافت درخواست، برقراری اتصال و دریافت پاسخ را مدیریت می‌کند. انتخاب Mongoose باعث شد بدون استفاده از یک Web Server خارجی، API مستقیماً در داخل برنامه C++ پیاده‌سازی شود.

فایل‌های پیکربندی

برای جلوگیری از ثابت‌نویسی IP، Port، Timeout و مسیر پایگاه داده، این اطلاعات در فایل config هر نود قرار گرفتند. Master علاوه بر Port و مسیر master.db، نشانی و Port هر دو Slave و مقدار SLAVE_TIMEOUT_MS را از فایل پیکربندی می‌خواند. مقدار Timeout در تست انجام‌شده ۳۰۰۰ میلی‌ثانیه بود. Slave‌ها نیز Port و مسیر پایگاه داده محلی خود را از فایل config دریافت می‌کنند. این طراحی باعث می‌شود پروژه بدون تغییر کد در شبکه یا ماشین دیگری اجرا شود و تنها فایل تنظیمات ویرایش گردد.

Makefile و اسکریپت‌های Bash

هر نود Makefile مستقل دارد. Makefile فایل‌های منبع C++ و فایل mongoose.c را کامپایل کرده و فایل اجرایی همان نود را ایجاد می‌کند. دستور make clean فایل‌های Object و فایل اجرایی قبلی را پاک می‌کند و دستور make یک Build کامل و تمیز انجام می‌دهد. اسکریپت‌های Bash پروژه برای مقداردهی اولیه پایگاه داده‌ها، کامپایل و اجرای نودها، بررسی پایگاه داده‌ها، ارسال درخواست‌های آزمایشی و مشاهده لاگ‌ها استفاده می‌شوند. وجود این اسکریپت‌ها باعث می‌شود مراحل تکراری پروژه به‌صورت قابل بازتولید انجام شوند و ارزیابی پروژه برای استاد یا آزمایش‌کننده ساده‌تر باشد.

طراحی پایگاه داده

هر نود یک فایل SQLite مستقل دارد. پایگاه داده Master با نام master.db، پایگاه داده Slave₁ با نام slave₁.db و پایگاه داده Slave₂ با نام slave₂.db ذخیره شده است. ساختار کلی پایگاه داده ها یکسان است و شامل جدول های node_info، sensors و sensor_readings می شود.

جدول node_info

جدول node_info برای ثبت هویت نود استفاده می شود. این جدول مشخص می کند فایل پایگاه داده متعلق به کدام نود است و نقش آن نود MASTER یا SLAVE است. وجود این جدول هنگام بررسی دستی پایگاه داده مفید است، زیرا از اشتباه گرفتن فایل های دیتابیس جلوگیری می کند.

جدول sensors

جدول sensors اطلاعات ثابت هر سنسور را نگهداری می کند. این اطلاعات شامل شناسه سنسور، نوع سنسور، نام سنسور، محل نصب و واحد اندازه گیری است. ترکیب شناسه و نوع سنسور در جست و جو استفاده می شود؛ بنابراین ارسال نوع اشتباه برای یک شناسه معتبر نباید به بازیابی داده منجر شود.

جدول sensor_readings

جدول sensor_readings مقادیر ثبت شده سنسورها را نگهداری می کند. هر رکورد شامل شناسه سنسور، مقدار خوانده شده و زمان ثبت است. برای یافتن آخرین مقدار، رکوردها ابتدا بر اساس recorded_at به صورت نزولی و در صورت برابر بودن زمان، بر اساس id به صورت نزولی مرتب می شوند. سپس اولین رکورد انتخاب می شود. این روش تضمین می کند جدیدترین Reading موجود به کاربر بازگردانده شود. برای افزایش سرعت جست و جو، روی نوع و شناسه سنسور و همچنین روی شناسه سنسور و زمان ثبت Reading ایندکس ایجاد شده است. اجرای Query ها نیز با Prepared Statement انجام می شود. این روش علاوه بر مدیریت بهتر پارامترها، خطر تزریق SQL را نسبت به ساختن مستقیم رشته Query کاهش می دهد.

مقدار دهی اولیه و توزیع داده ها

برای هر نود یک اسکریپت مقداردهی اولیه در نظر گرفته شد. این اسکریپت فایل داده مربوط به همان نود را خوانده، جدول های لازم را ایجاد کرده و داده های اولیه را وارد SQLite می کند. داده ها میان سه نود تقسیم شده اند و هر نود فقط بخشی از سنسورها و Reading ها را نگهداری می کند.

در بررسی انجام شده، هر پایگاه داده دارای چهار سنسور و هشت Reading بود. دستور PRAGMA integrity_check برای هر سه پایگاه داده مقدار ok را نمایش داد. این نتیجه نشان داد ساختار فایل های SQLite سالم است و خرابی داخلی در پایگاه داده مشاهده نشده است.

در Master، سنسور ۱۰۱ از نوع temperature قرار داشت و آخرین مقدار آن ۲۴.۸ بود. در Slave۱، سنسور ۲۰۴ از نوع co2 قرار داشت و آخرین مقدار آن ۷۳۵ ppm بود. در Slave۲، سنسور ۳۰۴ از نوع smoke قرار داشت و آخرین مقدار آن صفر بود. این سه سنسور برای بررسی مسیر جست و جو در هر سه نود انتخاب شدند.

بررسی پایگاه داده Master

```
amir@embedded-master:~/master$ sqlite3 -header -column slave1.db "SELECT * FROM node_info; SELECT COUNT(*) AS sensors_count FROM sensors; SELECT COUNT(*) AS readings_count FROM sensor_readings; PRAGMA integrity_check;"
Error: in prepare, no such table: node_info (1)
amir@embedded-master:~/master$ sqlite3 -header -column master.db "SELECT * FROM node_info; SELECT COUNT(*) AS sensors_count FROM sensors; SELECT COUNT(*) AS readings_count FROM sensor_readings; PRAGMA integrity_check;"
id node_name node_role description created_at
--
1 master MASTER Main node responsible for receiving operator requests 2026-07-14 11:27:08
sensors_count
-----
4
readings_count
-----
8
integrity_check
-----
ok
amir@embedded-master:~/master$ sqlite3 -header -column master.db "SELECT s.sensor_id,s.sensor_type,r.value,s.unit,r.recorded_at FROM sensors s JOIN sensor_readings r ON r.sensor_id=s.sensor_id WHERE r.id=(SELECT id FROM sensor_readings WHERE sensor_id=s.sensor_id ORDER BY recorded_at DESC,id DESC LIMIT 1) ORDER BY s.sensor_id;"
sensor_id sensor_type value unit recorded_at
-----
101 temperature 24.8 C 2026-06-01 10:15:00
102 humidity 45 % 2026-06-01 10:15:00
103 motion 1 bool 2026-06-01 10:16:00
104 temperature 23.9 C 2026-06-01 10:20:00
```

بررسی پایگاه داده Slave۱

```
amir@embedded-slave1:~/slave1$ sqlite3 -header -column slave1.db "SELECT * FROM node_info; SELECT COUNT(*) AS sensors_count FROM sensors; SELECT COUNT(*) AS readings_count FROM sensor_readings; PRAGMA integrity_check;"
id node_name node_role description created_at
--
1 slave1 SLAVE First slave node responsible for local sensor readings 2026-07-14 11:27:08
sensors_count
-----
4
readings_count
-----
8
integrity_check
-----
ok
amir@embedded-slave1:~/slave1$ sqlite3 -header -column slave1.db "SELECT s.sensor_id,s.sensor_type,r.value,s.unit,r.recorded_at FROM sensors s JOIN sensor_readings r ON r.sensor_id=s.sensor_id WHERE r.id=(SELECT id FROM sensor_readings WHERE sensor_id=s.sensor_id ORDER BY recorded_at DESC,id DESC LIMIT 1) ORDER BY s.sensor_id;"
sensor_id sensor_type value unit recorded_at
-----
201 temperature 25.3 C 2026-06-01 10:15:00
202 humidity 50 % 2026-06-01 10:15:00
203 motion 0 bool 2026-06-01 10:16:00
204 co2 735 ppm 2026-06-01 10:20:00
```


بررسی پایگاه داده Slave۲

```

amir@embedded-slave2:~/slave2$ sqlite3 -header -column slave2.db "SELECT * FROM node_info; SELECT COUNT(*) AS readings_count FROM sensor_readings; PRAGMA integrity_check;"
id node_name node_role description created_at
-----
1 slave2 SLAVE Second slave node responsible for local sensor readings 2026-07-14 11:27:08
sensors_count
-----
4
readings_count
-----
8
integrity_check
-----
ok
amir@embedded-slave2:~/slave2$ sqlite3 -header -column slave2.db "SELECT s.sensor_id,s.sensor_type,r.value,s.unit,r.recorded_at FROM sensors s JOIN sensor_readings r ON r.sensor_id=s.sensor_id WHERE r.id=(SELECT id FROM sensor_readings WHERE sensor_id=s.sensor_id ORDER BY recorded_at DESC, id DESC LIMIT 1) ORDER BY s.sensor_id;"
sensor_id sensor_type value unit recorded_at
-----
301 temperature 23.1 C 2026-06-01 10:15:00
302 humidity 41 % 2026-06-01 10:15:00
303 motion 0 bool 2026-06-01 10:16:00
304 smoke 0 bool 2026-06-01 10:20:00

```

مسیر درخواست و پاسخ

هنگام دریافت درخواست، Master ابتدا بررسی می کند که مسیر برابر با `api/sensor/` باشد. سپس وجود و خالی نبودن پارامترهای `id` و `type` را کنترل می کند. پس از اعتبارسنجی، پایگاه داده محلی Master جستجو می شود.

اگر داده در `master.db` وجود داشته باشد، Master بلافاصله پاسخ ۲۰۰ را ارسال می کند. در این حالت هیچ ارتباطی با Slaveها برقرار نمی شود. اگر داده در Master وجود نداشته باشد، درخواست HTTP به Slave۱ ارسال می شود. اگر Slave۱ پاسخ ۲۰۰ بدهد، Master همان داده را با `source` برابر Slave۱ به اپراتور می فرستد. اگر Slave۱ پاسخ ۴۰۴ بدهد، Master درخواست را برای Slave۲ ارسال می کند. اگر داده در Slave۲ پیدا شود، پاسخ نهایی با `source` برابر Slave۲ تولید می شود.

اگر هر سه نود با موفقیت جستجو شوند ولی داده در هیچ کدام وجود نداشته باشد، Master پاسخ ۴۰۴ برمی گرداند. تفاوت این حالت با خطای ارتباطی در این است که در پاسخ ۴۰۴ جستجوی توزیع شده کامل شده، اما داده پیدا نشده است. اگر یکی از نودهای لازم در دسترس نباشد و به همین علت نتوان با اطمینان گفت داده در کل سامانه وجود ندارد، پاسخ ۵۰۳ تولید می شود.

مدیریت خطا و کدهای وضعیت HTTP

پاسخ ۲۰۰ به معنای پیدا شدن Reading و بازگشت موفق داده است. پاسخ ۴۰۰ زمانی استفاده می شود که پارامتر `id` یا `type` در درخواست وجود نداشته باشد یا مقدار آن خالی باشد. پاسخ ۴۰۴ دو کاربرد دارد: یکی برای مسیر API ناشناخته و دیگری زمانی که پس از بررسی موفق هر سه نود، Reading موردنظر پیدا نشود. پاسخ ۵۰۰ برای خطای داخلی SQLite در سمت Slave پیش بینی شده است. پاسخ ۵۰۳ زمانی استفاده می شود که Master به دلیل خطای پایگاه داده، قطع ارتباط، یا پاسخ غیرمنتظره یک Slave نتواند جستجوی توزیع شده را کامل کند.

تفکیک ۴۰۴ و ۵۰۳ یک تصمیم مهم در طراحی بود. اگر یک Slave قطع باشد و Master مستقیماً ۴۰۴ برگرداند، اپراتور تصور می کند داده قطعاً وجود ندارد، در حالی که نود قطع شده بررسی نشده است. پاسخ ۵۰۳ به درستی نشان می دهد سرویس در آن لحظه قادر به ارائه نتیجه قطعی نیست.

مراحل کامپایل و اجرای برنامه

پس از بررسی وابستگی ها، روی هر سه نود ابتدا `make clean` و سپس `make` اجرا شد. فایل های اجرایی `master`، `slave1` و `slave2` با موفقیت ساخته شدند. هیچ خطای کامپایل مشاهده نشد و بررسی فایل های خروجی نشان داد که هر سه فایل از نوع اجرایی ۶۴-bit ELF هستند.

Master نود Build

```
amir@embedded-master:~/master$ make clean
rm -f master main.o http_client.o mongoose.o
amir@embedded-master:~/master$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c http_client.cpp -o http_client.o
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o http_client.o mongoose.o -lsqlite3 -o master
amir@embedded-master:~/master$ ls -lh master
-rwxrwxr-x 1 amir amir 219K Jul 18 13:05 master
amir@embedded-master:~/master$
```

Slave۱ نود Build

```
amir@embedded-slave1:~/slave1$ make clean
rm -f slave1 main.o mongoose.o
amir@embedded-slave1:~/slave1$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o mongoose.o -lsqlite3 -o slave1
amir@embedded-slave1:~/slave1$ ls -lh slave1
-rwxrwxr-x 1 amir amir 210K Jul 18 13:05 slave1
amir@embedded-slave1:~/slave1$
```

Slave۲ نود Build

```
amir@embedded-slave2:~/slave2$ make clean
rm -f slave2 main.o mongoose.o
amir@embedded-slave2:~/slave2$ make
g++ -std=c++17 -O2 -Wall -Wextra -Wpedantic -I../mongoose -c main.cpp -o main.o
gcc -O2 -Wall -Wextra -I../mongoose -c ../mongoose/mongoose.c -o mongoose.o
g++ main.o mongoose.o -lsqlite3 -o slave2
amir@embedded-slave2:~/slave2$ ls -lh slave2
-rwxrwxr-x 1 amir amir 210K Jul 18 13:06 slave2
amir@embedded-slave2:~/slave2$
```

برای اجرای سامانه ابتدا `Slave1`، سپس `Slave2` و در پایان `Master` اجرا شد. این ترتیب باعث شد زمانی که `Master` آغاز به کار می کند، هر دو سرویس مقصد آماده پاسخگویی باشند. `Slave1` روی `Port 9001`، `Slave2` روی `Port 9002` و `Master` روی `Port 8000` در حالت `Listening` قرار گرفتند.

اجرای Slave۱

```
amir@embedded-slave1:~/slave1$ ./slave1 config
10fe90a 3 mongoose.c:7806:mg_mgr_init MG IO SIZE: 16384, TLS: none
10fe90a 3 mongoose.c:7719:mg_listen 1 4 http://0.0.0.0:9001
Slave running on port 9001, database: slave1.db
```

اجرای Slave۲

```
amir@embedded-slave2:~/slave2$ ./slave2 config
10ea9ef 3 mongoose.c:7806:mg_mgr_init   MG IO SIZE: 16384, TLS: none
10ea9f0 3 mongoose.c:7719:mg_listen     1 4 http://0.0.0.0:9002
Slave running on port 9002, database: slave2.db
```

اجرای Master

```
amir@embedded-master:~/master$ ./master config
111c19f 3 mongoose.c:7806:mg_mgr_init   MG IO SIZE: 16384, TLS: none
111c1a0 3 mongoose.c:7719:mg_listen     1 4 http://0.0.0.0:8000
Master running on port 8000, database: master.db
Slave1: 192.168.122.18:9001
Slave2: 192.168.122.190:9002
```

در ادامه با استفاده از netcat باز بودن Port های 8000، 9001 و 9002 از سیستم اصلی بررسی شد. نتیجه هر سه اتصال موفق بود و نشان داد سرویس ها از طریق شبکه در دسترس هستند.

بررسی Port های در حال Listening

```
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ MASTER_IP=192.168.122.22
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ nc -zv $MASTER_IP 8000
Connection to 192.168.122.22 8000 port [tcp/*] succeeded!
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ nc -zv 192.168.122.18 9001
Connection to 192.168.122.18 9001 port [tcp/*] succeeded!
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ nc -zv 192.168.122.190 9002
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```

روش انجام آزمون‌ها

آزمون‌ها از سیستم اصلی و با استفاده از ترمینال چهارم انجام شدند. سه ترمینال دیگر برای مشاهده هم‌زمان خروجی Master، Slave۱ و Slave۲ باز نگه داشته شدند. در هر تست، علاوه بر پاسخ curl، لاگ Master نیز بررسی شد تا مشخص شود درخواست محلی پاسخ داده شده یا برای Slaveها ارسال شده است.

آزمون ارتباط شبکه

پیش از اجرای برنامه، ارتباط IP میان Master و هر دو Slave با دستور ping بررسی شد. همچنین از Slaveها نیز ارتباط با Master کنترل شد. دریافت پاسخ موفق از هر سه ماشین نشان داد تنظیمات شبکه صحیح است و Packet Loss مؤثری در زمان تست وجود ندارد.

```
amir@embedded-master:~/master$ ping -c 3 192.168.122.18
PING 192.168.122.18 (192.168.122.18) 56(84) bytes of data.
64 bytes from 192.168.122.18: icmp_seq=1 ttl=64 time=0.526 ms
64 bytes from 192.168.122.18: icmp_seq=2 ttl=64 time=0.702 ms
64 bytes from 192.168.122.18: icmp_seq=3 ttl=64 time=0.854 ms

--- 192.168.122.18 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2033ms
rtt min/avg/max/mdev = 0.526/0.694/0.854/0.134 ms
amir@embedded-master:~/master$ ping -c 3 192.168.122.190
PING 192.168.122.190 (192.168.122.190) 56(84) bytes of data.
64 bytes from 192.168.122.190: icmp_seq=1 ttl=64 time=0.546 ms
64 bytes from 192.168.122.190: icmp_seq=2 ttl=64 time=0.791 ms
64 bytes from 192.168.122.190: icmp_seq=3 ttl=64 time=0.825 ms

--- 192.168.122.190 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2037ms
rtt min/avg/max/mdev = 0.546/0.720/0.825/0.124 ms
```

آزمون بازیابی داده از Master

در اولین آزمون، درخواست سنسور 101 از نوع temperature به Master ارسال شد. این سنسور در master.db قرار داشت. پاسخ HTTP با کد 200 دریافت شد، مقدار source برابر master بود و آخرین مقدار سنسور برابر 24.8 بازگردانده شد. در این حالت Master بدون ارسال درخواست به Slaveها نتیجه را تولید کرد. این آزمون نشان داد جست‌وجوی محلی و اصل Local First به‌درستی پیاده‌سازی شده است.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=101&type=temperature"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 201

{"source": "master", "data": {"sensor_id": "101", "sensor_type": "temperature", "sensor_name": "Floor1 Room101_Temp", "location": "Floor1_Room101", "value": "24.8", "unit": "C", "recorded_at": "2026-06-01 10:15:00"}}
```

آزمون بازیابی داده از Slave۱

در آزمون دوم، سنسور ۲۰۴ از نوع CO2 درخواست شد. این سنسور در Master وجود نداشت و در slave۱.db نگهداری می شد. Master پس از ناموفق بودن جست و جوی محلی، درخواست را به ۱۹۲.۱۶۸.۱۲۲.۱۸ روی Port ۹۰۰۱ ارسال کرد. Slave۱ آخرین مقدار ثبت شده، یعنی ۷۳۵ ppm، را برگرداند. پاسخ نهایی با کد ۲۰۰ و source برابر slave۱ به اپراتور ارائه شد. این نتیجه مسیر Master به Slave۱ و بازگشت پاسخ از Slave۱ به Master را تأیید کرد.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=204&type=co2"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 198

{"source":"slave1","data":{"sensor_id":"204","sensor_type":"co2","sensor_name":"Floor2_Meeting_CO2","location":"Floor2_MeetingRoom","value":"735","unit":"ppm","recorded_at":"2026-06-01 10:20:00"}}
```

آزمون بازیابی داده از Slave۲

در آزمون سوم، سنسور ۳۰۴ از نوع smoke درخواست شد. این داده نه در Master و نه در Slave۱ وجود داشت، اما در Slave۲ ذخیره شده بود. Master ابتدا پایگاه داده محلی خود را بررسی کرد، سپس از Slave۱ پاسخ ۴۰۴ دریافت کرد و در مرحله بعد درخواست را به ۱۹۲.۱۶۸.۱۲۲.۱۹۰ روی Port ۹۰۰۲ فرستاد. Slave۲ مقدار صفر را برگرداند و پاسخ نهایی با کد ۲۰۰ و source برابر slave۲ ارائه شد. این تست کامل ترین مسیر عادی سامانه را بررسی کرد و نشان داد زنجیره جست و جوی ترتیبی به درستی اجرا می شود.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=304&type=smoke"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 197

{"source":"slave2","data":{"sensor_id":"304","sensor_type":"smoke","sensor_name":"Floor3_Storage_Smoke","location":"Floor3_Storage","value":"0","unit":"bool","recorded_at":"2026-06-01 10:20:00"}}
```

آزمون نبود سنسور در تمام نودها

برای بررسی حالت Not Found، شناسه ۹۹۹ از نوع temperature ارسال شد. این داده در هیچ یک از سه پایگاه داده وجود نداشت. Master هر سه محل را با موفقیت بررسی کرد و در پایان پاسخ ۴۰۴ را با پیام مربوط به پیدا نشدن Reading در هیچ نود بازگرداند. این تست نشان داد پاسخ Not Found فقط پس از تکمیل جست و جوی کل سامانه تولید می شود.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=999&type=temperature"
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 49

{"error":"sensor reading not found in any node"}
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```

آزمون نبود پارامتر ورودی

در این آزمون، درخواست فقط با پارامتر id و بدون type ارسال شد. Master پیش از دسترسی به پایگاه داده، ورودی را نامعتبر تشخیص داد و پاسخ 400 Bad Request را تولید کرد. این رفتار از اجرای Query ناقص جلوگیری می‌کند و پیام واضحی درباره پارامترهای موردنیاز به کاربر می‌دهد.

```
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=101"
HTTP/1.1 400 Bad Request
Content-Type: application/json
Content-Length: 54

{"error": "id and type query parameters are required"}
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```

آزمون مسیر API نامعتبر

درخواست دیگری به مسیر api/unknown ارسال شد. چون این مسیر در برنامه تعریف نشده بود، پاسخ 404 مربوط به Route نامعتبر دریافت شد. این آزمون نشان داد برنامه میان نبود منبع داده و ناشناخته بودن مسیر HTTP تمایز منطقی ایجاد کرده و برای مسیرهای خارج از API پاسخ کنترل شده می‌دهد.

```
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/unknown"
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 28

{"error": "route not found"}
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```

آزمون ناسازگاری شناسه و نوع سنسور

برای بررسی اینکه جست‌وجو فقط بر اساس شناسه انجام نمی‌شود، شناسه معتبر ۱۰۱ با نوع اشتباه humidity ارسال شد. سنسور ۱۰۱ در پایگاه داده وجود داشت، اما نوع واقعی آن temperature بود. نتیجه نهایی 404 شد. این خروجی تأیید کرد که برنامه هر دو پارامتر sensor_id و sensor_type را در Query لحاظ می‌کند و سنسور با نوع نادرست پذیرفته نمی‌شود.

```
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=101&type=humidity"
HTTP/1.1 404 Not Found
Content-Type: application/json
Content-Length: 49

{"error": "sensor reading not found in any node"}
amir-hosseini-motiei@amir-hosseini-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```


آزمون دسترسی مستقیم به Slaveها

برای بررسی وضعیت امنیت شبکه، از سیستم اصلی مستقیماً به Portهای Slave ۱ و Slave ۲ درخواست ارسال شد. درخواست مستقیم سنسور ۲۰۴ به Slave ۱ و درخواست مستقیم سنسور ۳۰۴ به Slave ۲ هر دو پاسخ ۲۰۰ دریافت کردند. از نظر عملکرد داخلی برنامه، این پاسخها صحیح بودند، زیرا سرویس Slave توانست پایگاه داده محلی خود را بخواند. با این حال، از نظر معماری مورد انتظار، اپراتور باید فقط با Master ارتباط داشته باشد.

این تست نشان داد که در نسخه فعلی، برنامههای Slave روی ۰.۰.۰.۰ گوش می‌دهند و قانون فایروال محدودکننده‌ای روی ماشین‌های مجازی اعمال نشده است. بنابراین Portهای ۹۰۰۱ و ۹۰۰۲ از سیستم اصلی قابل دسترسی بودند. این مورد به‌عنوان یک محدودیت امنیتی شناسایی شد. راه‌حل مناسب این است که روی Slave ۱ فقط IP نود Master اجازه اتصال به Port ۹۰۰۱ را داشته باشد و روی Slave ۲ نیز فقط IP Master اجازه اتصال به Port ۹۰۰۲ را دریافت کند. در نتیجه، منطق اصلی برنامه صحیح است، اما برای انطباق کامل در سطح شبکه باید محدودیت فایروال اعمال شود.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ SLAVE1_IP=192.168.122.18
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ SLAVE2_IP=192.168.122.190
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$SLAVE1_IP:9001/api/sensor?id=204&type=co2"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 170

{"sensor_id":"204","sensor_type":"co2","sensor_name":"Floor2_Meeting_CO2","location":"Floor2_MeetingRoom","value":"735","unit":"ppm","recorded_at":"2026-06-01 10:20:00"}
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$SLAVE2_IP:9002/api/sensor?id=304&type=smoke"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 169

{"sensor_id":"304","sensor_type":"smoke","sensor_name":"Floor3_Storage_Smoke","location":"Floor3_Storage"
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```

آزمون ادامه سرویس با قطع Slave ۱

برای بررسی تحمل خطا، برنامه Slave ۱ به‌صورت موقت متوقف شد. سپس سنسور ۳۰۴ که در Slave ۲ قرار داشت از طریق Master درخواست شد. Master ابتدا جست‌وجوی محلی را انجام داد، سپس تلاش برای ارتباط با Slave ۱ ناموفق بود، اما فرایند را متوقف نکرد و درخواست را به Slave ۲ فرستاد. Slave ۲ داده را با موفقیت برگرداند و اپراتور پاسخ ۲۰۰ دریافت کرد. این آزمون نشان داد قطع Slave ۱ باعث از دسترس خارج شدن کامل سامانه نمی‌شود و اگر داده موردنظر در نود بعدی وجود داشته باشد، Master می‌تواند جست‌وجو را ادامه دهد. پس از پایان تست، Slave ۱ دوباره اجرا شد.

```
-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$ curl -i "http://$MASTER_IP:8000/api/sensor?id=304&type=smoke"
HTTP/1.1 200 OK
Content-Type: application/json
Content-Length: 197

{"source":"slave2","data":{"sensor_id":"304","sensor_type":"smoke","sensor_name":"Floor3_Storage_Smoke","location":"Floor3_Storage","value":"0","unit":"bool","recorded_at":"2026-06-01 10:20:00"}}
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01$
```


آزمون پاسخ ۵۰۳ هنگام قطع نود دارنده داده

در آزمون بعدی، Slave₂ متوقف شد و سنسور ۳۰۴ از طریق Master درخواست گردید. این سنسور در Master و Slave₁ وجود نداشت و نود دارنده داده، یعنی Slave₂، در دسترس نبود. در این شرایط Master نمی توانست نتیجه قطعی Not Found تولید کند، زیرا جست و جوی توزیع شده کامل نشده بود. بنابراین پاسخ ۵۰۳ Service Unavailable با پیام distributed query could not be completed بازگردانده شد. این نتیجه نشان داد پیاده سازی میان «داده وجود ندارد» و «امکان بررسی همه نودها وجود ندارد» تفاوت قائل می شود. این تمایز برای سامانه های توزیع شده مهم است و از ارائه پاسخ اشتباه به اپراتور جلوگیری می کند. پس از ثبت خروجی، Slave₂ مجدداً راه اندازی شد.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01/scripts$ curl -i "http://192.168.122.22:8000/api/sensor?id=304&type=smoke"
HTTP/1.1 503 Service Unavailable
Content-Type: application/json
Content-Length: 53

amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01/scripts$
```

آزمون خودکار نهایی

پس از راه اندازی مجدد هر سه نود، اسکریپت test_requests.sh از سیستم اصلی اجرا شد. نشانی Master از طریق متغیر MASTER_HOST در اختیار اسکریپت قرار گرفت. این اسکریپت شش سناریوی اصلی شامل بازیابی از Master، بازیابی از Slave₁، بازیابی از Slave₂ نبود داده در همه نودها، نبود پارامتر type و مسیر API ناشناخته را به صورت خودکار اجرا کرد. خلاصه نهایی اسکریپت برابر با ۶ Passed: و ۰ Failed: بود و پیام All tests passed successfully نمایش داده شد. این نتیجه نشان داد تمام سناریوهای تعریف شده در تست خودکار با موفقیت اجرا شده اند و پس از تست های قطع و وصل نودها، سامانه دوباره به حالت عملیاتی کامل بازگشته است.

```
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01/scripts$ MASTER_HOST=192.168.122.22 bash test_requests.sh

Test: Reading found in Master
Request: GET http://192.168.122.22:8000/api/sensor?id=101&type=temperature
HTTP status: 200
Response:
{"source":"master","data":{"sensor_id":"101","sensor_type":"temperature","sensor_name":"Floor1_Room101_Temp","location":"Floor1_Room101","value":"24.8","unit":"C","recorded_at":"2026-06-01 10:15:00"}}
Result: PASSED

=====
Test: Reading found in Slave1 through Master
Request: GET http://192.168.122.22:8000/api/sensor?id=204&type=co2
HTTP status: 200
Response:
{"source":"slave1","data":{"sensor_id":"204","sensor_type":"co2","sensor_name":"Floor2_Meeting_CO2","location":"Floor2_MeetingRoom","value":"735","unit":"ppm","recorded_at":"2026-06-01 10:20:00"}}
Result: PASSED

=====
Test: Reading found in Slave2 through Master
Request: GET http://192.168.122.22:8000/api/sensor?id=304&type=smoke
HTTP status: 200
Response:
{"source":"slave2","data":{"sensor_id":"304","sensor_type":"smoke","sensor_name":"Floor3_Storage_Smoke","location":"Floor3_Storage","value":"0","unit":"bool","recorded_at":"2026-06-01 10:20:00"}}
Result: PASSED

=====
Test: Reading not found in any node
Request: GET http://192.168.122.22:8000/api/sensor?id=999&type=temperature
HTTP status: 404
Response:
{"error":"sensor reading not found in any node"}
Result: PASSED

=====
Test: Invalid request with missing type
Request: GET http://192.168.122.22:8000/api/sensor?id=101
HTTP status: 400
Response:
{"error":"id and type query parameters are required"}
Result: PASSED

=====
Test: Unknown API route
Request: GET http://192.168.122.22:8000/api/unknown
HTTP status: 404
Response:
{"error":"route not found"}
Result: PASSED

=====
Test summary
Passed: 6
Failed: 0
amir-hossein-motiei@amir-hossein-motiei-GP66-Leopard-11UG:~/embedded/embedded_hw04/01/scripts$
```

تحلیل نتایج آزمون‌ها

نتایج آزمون‌ها نشان دادند که بخش اصلی معماری توزیع شده مطابق طراحی عمل می‌کند. در درخواست سنسور ۱۰۱، داده بدون ایجاد ترافیک میان نودها از Master خوانده شد. در درخواست سنسور ۲۰۴، Master توانست نبود داده محلی را تشخیص دهد و پاسخ را از Slave ۱ دریافت کند. در درخواست سنسور ۳۰۴ نیز مسیر کامل Master، Slave ۱ و Slave ۲ طی شد و داده از آخرین نود به اپراتور رسید.

پاسخ‌های خطا نیز قابل تفکیک و معنادار بودند. ورودی ناقص به ۴۰۰، مسیر ناشناخته به ۴۰۴، نبود واقعی داده در همه نودها به ۴۰۴ و ناقص ماندن جست‌وجوی توزیع شده به ۵۰۳ منجر شد. این رفتار نشان می‌دهد خطاها صرفاً با یک پاسخ عمومی مدیریت نشده‌اند و وضعیت واقعی سامانه در کد HTTP منعکس می‌شود.

تست قطع Slave ۱ نشان داد طراحی در برابر از دسترس خارج شدن یک نود میانی تا حدی مقاوم است و جست‌وجو می‌تواند در نود بعدی ادامه یابد. تست قطع Slave ۲ نیز نشان داد در صورت عدم امکان بررسی نود دارنده داده، سیستم پاسخ نادرست Not Found تولید نمی‌کند. از سوی دیگر، تست دسترسی مستقیم به Slave ها یک ضعف تنظیماتی در سطح شبکه را آشکار کرد که باید با Firewall یا محدود کردن Bind Address اصلاح شود.

بررسی امنیت ارتباط HTTP

نسخه فعلی برای محیط آموزشی و شبکه خصوصی طراحی شده و ارتباطات آن با HTTP ساده انجام می‌شود. در نتیجه داده‌ها رمزگذاری نمی‌شوند و سازوکار احراز هویت برای اپراتور یا ارتباط بین نودها وجود ندارد. هر سیستمی که به شبکه دسترسی داشته باشد و محدودیت فایروال نداشته باشد، ممکن است بتواند درخواست‌ها یا پاسخ‌ها را مشاهده کند یا مستقیماً به Slave های متصل شود.

در عین حال، چند اقدام پایه امنیتی در پیاده‌سازی وجود دارد. پارامترهای ورودی قبل از Query بررسی می‌شوند، Query های SQLite با Prepared Statement اجرا می‌شوند، Timeout ارتباط Slave قابل تنظیم است، مسیرهای ناشناخته پاسخ مشخص دریافت می‌کنند و IP، Port و مسیر دیتابیس از فایل تنظیمات خوانده می‌شوند.

برای نسخه کامل‌تر، استفاده از HTTPS و TLS برای رمزگذاری ارتباط اپراتور با Master و ارتباط Master با Slave ها پیشنهاد می‌شود. همچنین می‌توان از API Key یا Token برای احراز هویت اپراتور و از Mutual TLS یا Shared Secret برای احراز هویت نودها استفاده کرد. مهم‌ترین اصلاح فوری در شبکه فعلی، اعمال قانون فایروال است تا Port ۹۰۰۱ در Slave ۱ و Port ۹۰۰۲ در Slave ۲ فقط از ۱۹۲.۱۶۸.۱۲۲.۲۲ IP قابل دسترسی باشند. محدود کردن دسترسی Port ۸۰۰۰ به شبکه مورد اعتماد، افزودن Rate Limiting، تعیین حداکثر اندازه درخواست، تنظیم سطح دسترسی فایل‌های config و db و جلوگیری از ثبت اطلاعات حساس در لاگ‌ها نیز پیشنهاد می‌شود.

تصمیم‌های طراحی و دلایل آن‌ها

انتخاب SQLite به دلیل سبک بودن، عدم نیاز به سرویس جداگانه و سادگی استقرار روی هر ماشین مجازی انجام شد. این پایگاه داده برای پروژه آموزشی و حجم کم داده‌های سنسور مناسب است. انتخاب Mongoose نیز به دلیل امکان پیاده‌سازی مستقیم HTTP Server و Client در C و C++ و ساختار رویدادمحور آن صورت گرفت.

جستوجوی ترتیبی به جای جستوجوی همزمان انتخاب شد، زیرا ترتیب محل داده در صورت تمرین مشخص بود و این روش پیاده سازی و تحلیل ساده تری داشت. قرار دادن Master به عنوان تنها نقطه ورود منطقی باعث شد جزئیات توزیع داده از اپراتور پنهان بماند. استفاده از فایل پیکربندی نیز امکان تغییر IP، Port، Timeout و مسیر دیتابیس را بدون کامپایل مجدد فراهم کرد.

افزودن مقدار source به پاسخ Master تصمیمی برای افزایش قابلیت مشاهده و تست پذیری بود. این فیلد در یک سامانه واقعی می تواند از پاسخ عمومی حذف شود، اما در محیط آزمایش کمک کرد محل واقعی داده و مسیر طی شده به وضوح مشخص شود.

محدودیت های نسخه فعلی

نسخه فعلی فاقد Replication است و هر سنسور تنها در یک نود قرار دارد. بنابراین اگر نود دارنده داده قطع شود، نسخه جایگزینی از داده در نود دیگر وجود ندارد. Master نیز یک نقطه خرابی منفرد است؛ در صورت قطع شدن Master، اپراتور نمی تواند از مسیر اصلی به داده ها دسترسی داشته باشد. جستوجوی ترتیبی با افزایش تعداد نودها می تواند زمان پاسخ را بیشتر کند و برای سامانه بزرگ نیاز به مکانیزم کشف محل داده خواهد داشت.

محدودیت مهم دیگر، دسترسی مستقیم شبکه به Slave ها است که در آزمون مشاهده شد. این مسئله با تغییر منطق برنامه مرتبط نیست و از طریق تنظیم فایروال و جداسازی شبکه قابل رفع است. HTTP ساده، نبود احراز هویت و نبود رمزگذاری نیز از دیگر محدودیت های نسخه آموزشی هستند.

پیشنهادهای توسعه آینده

در ادامه پروژه می توان Replication داده را اضافه کرد تا خرابی یک Slave باعث عدم دسترسی به سنسورهای آن نشود. استفاده از Cache می تواند تعداد مراجعه ها به SQLite و زمان پاسخ را کاهش دهد. طراحی Health Check دوره ای برای نودها، ثبت Metric هایی مانند زمان پاسخ و تعداد خطاها، افزودن Structured Logging و ایجاد داشبورد مانیتورینگ نیز قابلیت نگهداری سامانه را افزایش می دهد.

در معماری بزرگ تر می توان به جای ترتیب ثابت، یک جدول یا سرویس Metadata برای مشخص کردن محل هر سنسور ایجاد کرد. همچنین امکان استفاده از Master، Load Balancer، پشتیبان، Replication پایگاه داده و پیام رسانی های مانند MQTT برای دریافت داده های سنسورها وجود دارد.